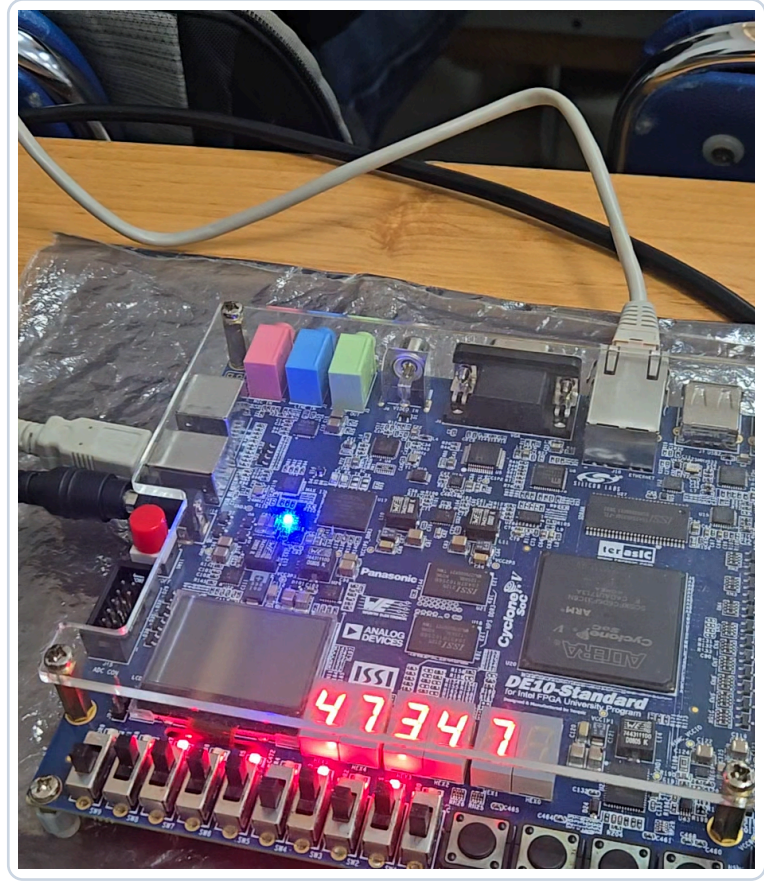


SoC Ethernet Controller

DE10-Standard Cyclone V SoC FPGA
TCP control path from PC clients to FPGA HEX displays

Presentation information

Course Embedded Systems | Group 17 | 22DTV_CLC
Team Five members; names are listed in the technical report
Duration 10-15 minutes



Main message

This is not a collection of separate modules.

The presentation follows one verifiable end-to-end control path:

Technical path

PC GUI -> **TCP** -> **HPS Linux** -> **Bridge** -> **HEX**

1

end-to-end control path

6

maximum characters on
HEX0..HEX5

5000

TCP service port on the board

LW

bridge base at 0xFF200000

Problem and scope

Input

- The user supplies a board IP address, TCP port, and payload of up to six characters.
- A client sends the payload through a TCP socket on the local LAN.
- The board returns OK:<text> or ERR:<reason>.

Deliberately bounded scope

- Local LAN only; no Internet or NAT traversal.
- Functional integration, not measured latency or throughput targets.
- A simple text-line protocol; no len/cmd/crc frame.
- devmem for the prototype; no custom kernel driver.

Output

- The normalized string appears on HEX5..HEX0.
- The PC log records each request and response.
- An Android client uses the same service to validate an independent endpoint.

Prototype value

Each layer has observable evidence: a Linux shell, eth0 address, TCP socket, OK/ERR response, and physical HEX state.

System architecture

PC GUI

Enter payload, send TCP, store log

TCP

STREAM socket, one request-response connection

HPS Linux

Python service listens on 0.0.0.0:5000

Sanitize

Uppercase, filter characters, crop or pad to six

Bridge

devmem writes the PIO registers

HEX

HEX5..HEX0, displayed left to right

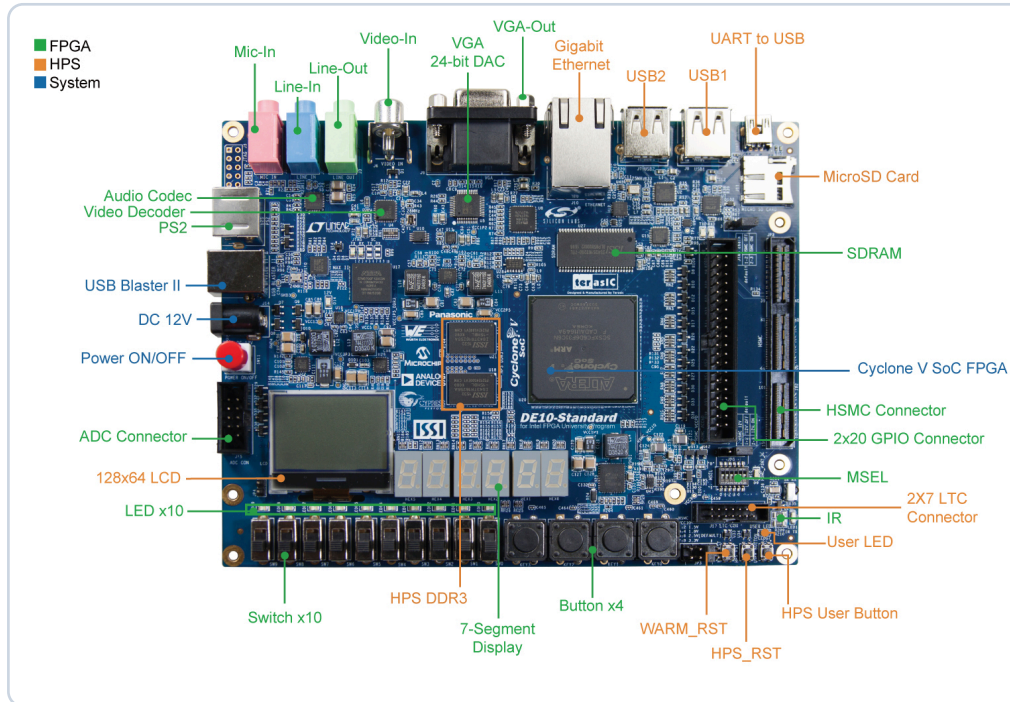
Design approach

Each layer has an input, output, and test method. A failure is isolated at the first broken link instead of guessed across the entire system.

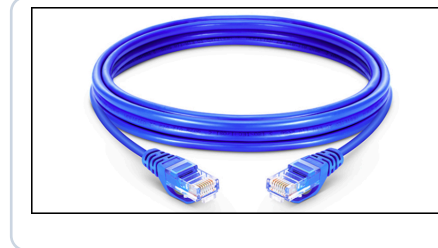
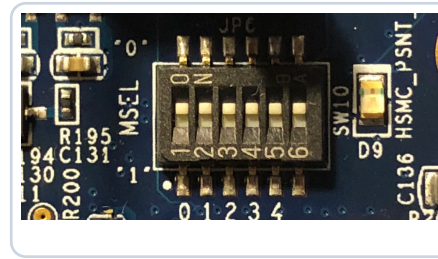
Speaking point

The Android client is not decoration. It shows that the board TCP service is not coupled to the PC GUI.

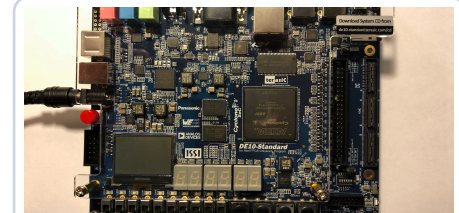
Hardware platform and bring-up



DE10-Standard Cyclone V SoC FPGA



Ethernet LAN



USB-UART, Ethernet, and power

Minimum bring-up

1. Load the bitstream and boot files.
2. Boot Linux from MicroSD and open the shell through USB-UART.
3. Enable eth0 and request an address by DHCP.
4. Run the TCP server and test it from a PC on the LAN.

Platform Designer map

Use	Connections	Name	Description	Export	Clock	Base	End	IRQ	Tags	Opcode Name
☒		clk_0	Clock Source							
		clk_in	Clock Input							
		clk_in_reset	Reset Input	clk_reset	exported					
		clk	Clock Output	Double-click to	clk_0					
		clk_reset	Reset Output	Double-click to						
☒		hps_0	Arria V/Cyclone V Hard Proce...							
		memory	Conduit	hps_0 ddr						
		hps_io	Conduit	hps_0 io						
		h2f_reset	Reset Output	Double-click to						
		f2h_sdram0_clock	Clock Input	Double-click to						
		f2h_sdram0_data	Avalon Memory Mapped Slave	Double-click to	pll_0 ou...	# 0x0000_0000	0xffff_ffff			
		f2h_sdram1_clock	Clock Input	Double-click to	pll_0 ou...	# 0x0000_0000	0xffff_ffff			
		f2h_sdram1_data	Avalon Memory Mapped Slave	Double-click to	pll_0 ou...	# 0x0000_0000	0xffff_ffff			
		h2f_lw_axi_clock	Clock Input	Double-click to	pll_0 ou...	# 0x0000_0000	0xffff_ffff			
		h2f_lw_axi_master	AXI Master	Double-click to	pll_0 ou...	# 0x0000_0000	0xffff_ffff			
☒		pio_0	Pio (Parallel I/O) Intel FPGA IP							
		clk	Clock Input	Double-click to	pll_0 ou...	# 0x0000_0000	0xffff_ffff			
		reset	Reset Input	Double-click to	[clk]					
		s1	Avalon Memory Mapped Slave	Double-click to	[clk]	# 0x0000_0030	0x0000_003f			
		external_connec...	Conduit	Double-click to	pio_leds_external					
☒		msgdma_0	Modular Scatter-Gather DMA L...							
		mm_read	Avalon Memory Mapped Master	Double-click to	[clock]					
		mm_write	Avalon Memory Mapped Master	Double-click to	[clock]					
		clock	Clock Input	Double-click to	pll_0 ou...	# 0x0000_0000	0x0000_001f			
		reset_n	Reset Input	Double-click to	[clock]					
		csr	Avalon Memory Mapped Slave	Double-click to	[clock]	# 0x0000_0000	0x0000_002f			
		descriptor_slave	Avalon Memory Mapped Slave	Double-click to	[clock]	# 0x0000_0020	0x0000_002f			
		csr_irq	Interrupt Sender	Double-click to	[clock]					
☒		pll_0	PLL Intel FPGA IP							
		refclk	Clock Input	Double-click to	clk_0					
		reset	Reset Input	Double-click to	pll_0_out...					
		outclk0	Clock Output	Double-click to						

Platform Designer system

Hardware-software contract

Top-level VHDL connects pio_hex0_external_export through pio_hex5_external_export to HEX0 through HEX5. HPS code writes the agreed PIO addresses.

pio_hex0

0xFF200040

HEX0

pio_hex1

0xFF200050

HEX1

pio_hex2

0xFF200060

HEX2

pio_hex3

0xFF200070

HEX3

pio_hex4

0xFF200080

HEX4

pio_hex5

0xFF200090

HEX5

Quartus result

Fitter successful. Logic utilization: **1,960 / 41,910 ALMs (5%)**. Total registers: **2,614**. Total pins: **302 / 499 (61%)**.

HPS Linux firmware

handle_client()

Receive `recv(1024)`, decode UTF-8, process one request, then close the connection.

sanitize_text()

Trim, uppercase, filter unsupported characters, then crop or pad to exactly six.

seg() + write_hex()

Encode active-low seven-segment values, then use `devmem` to write six PIO registers.

Active-low convention

0 turns a segment on and 1 turns it off. Therefore 8 = 0, while a blank is 127.

Server logic core

```
s.bind(("0.0.0.0", 5000))
s.listen(5)

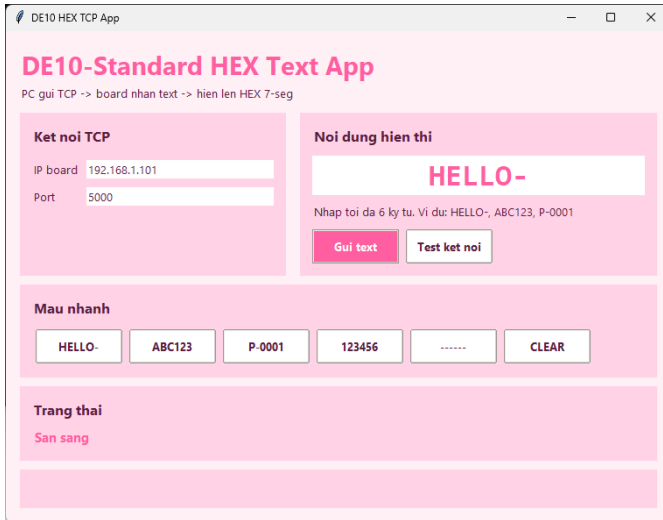
data = conn.recv(1024)
text = sanitize_text(data)

v5, v4, v3, v2, v1, v0 = map(seg, text[:6])
devmem 0xFF200040..0xFF200090 32 value

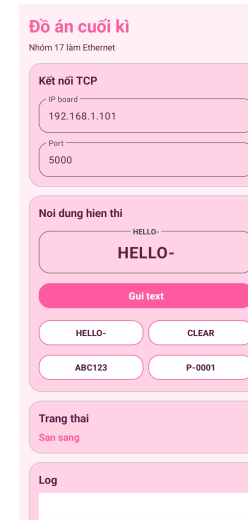
conn.sendall(("OK:%s\n" % text).encode())
```

For failures, `ERR:<reason>` distinguishes socket, `devmem`, root-permission, and address-map errors.

Client applications



Python/Tkinter PC GUI



Android client using the shared TCP service

PC GUI

- A dedicated TCP thread keeps the interface responsive.
- Five-second socket timeout.
- Clear sent, received, and error logs.

Android client

- Sends the same newline-terminated UTF-8 payload.
- Receives the same OK:<text> or ERR:<reason> response.
- Confirms that the server is independent of the endpoint.

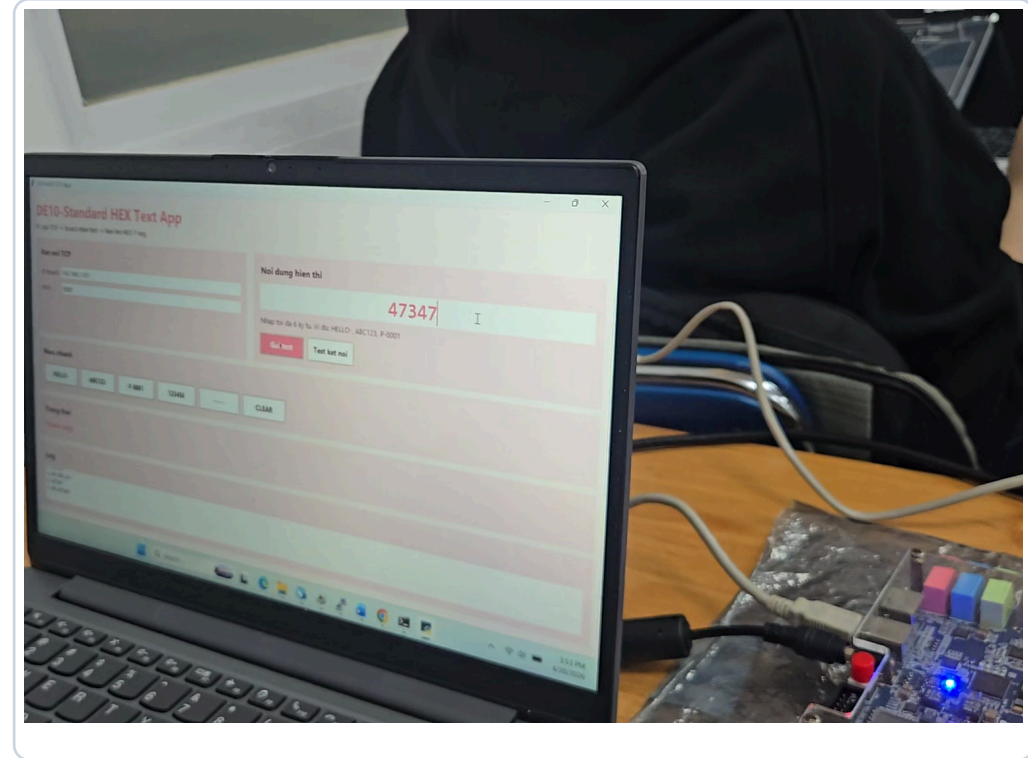
End-to-end test

Five-step procedure

1. Boot Linux and open the shell through USB-UART.
2. Enable eth0, request a DHCP address, and ping the network.
3. Run the TCP server on port 5000.
4. Send a payload from the PC GUI and receive OK:<text>.
5. Compare HEX5..HEX0 with the normalized string.

Test payloads

HELL0-, ABC123, P-0001, 123456, -----, an empty string, a string longer than six characters, and unsupported characters.



Laptop, router, and board test environment

Results and fault isolation



PC and Android controlling the TCP service

Fault-isolation tree

- No shell: check power, MSEL, MicroSD, and USB-UART.
- Shell but no IP: check Ethernet, DHCP, and udhcpc.
- IP but no connection: check the server and port 5000.
- OK but wrong HEX: check seg(), HEX5..HEX0 order, and VHDL.
- ERR: read the reason; check devmem, root permissions, and PIO addresses.

The team used a layer-by-layer debugging method, not a one-time successful demonstration.

Functional result

The PC/Android -> TCP -> HPS Linux -> PIO -> HEX path was repeatable on the LAN. Valid payloads receive 0K and are normalized before display.

Limits and next steps

Current limits

- The server does not start automatically after reboot.
- The protocol remains a simple text line.
- devmem suits the prototype but not a long-term product.
- No authentication, encryption, or command-protection mechanism.
- No quantitative latency or throughput measurements.

Next steps

- Start the server with init or systemd.
- Design a len/cmd/payload/crc frame.
- Replace devmem with mmap or a kernel driver.
- Add simple LAN authentication.
- Write automated tests for multiple payloads.

Response to deeper questions

These limits do not remove the project's value: the course objective is integration and verification of a hardware-software path. The team identifies what is prototype scope and what a production system would require.

Conclusion

The team built a SoC Ethernet prototype with a clear end-to-end control path that can be tested and explained at every layer.

Key point: PC or Android clients send TCP commands; HPS Linux processes them and writes through the lightweight HPS-to-FPGA bridge; FPGA fabric displays the result on HEX0..HEX5.

Thank you for your attention.